# Customer Churn Data Acquisition and Preprocessing Strategy

*Customer Churn Prediction Using Python — Week 2*

Data Engineering & Preprocessing Plan (Technical Internship Project)

Week 1 established the strategic framework and objectives for the Customer Churn Prediction project — that is, what the project intends to do. Week 2 moves from planning to execution readiness: it identifies the actual dataset to be used and documents exactly how that data will be obtained, validated, cleaned, and transformed before modelling begins in later weeks.

## 1. Data Sources

### 1.1 Selected Data Source

- **Dataset:** Telco Customer Churn Dataset
- **Provider:** IBM Sample Data Sets, hosted publicly on Kaggle
- **Format:** Single CSV file (WA_Fn-UseC_-Telco-Customer-Churn.csv), approximately 7,043 rows and 21 columns
- **Access method:** Downloaded directly from Kaggle (kaggle.com/datasets) / IBM's public sample data repository, and committed to this project's GitHub repository at Week-2/data/WA_Fn-UseC_-Telco-Customer-Churn.csv for reproducibility

### 1.2 Why This Source Was Selected

- It is a real-world telecom dataset with a genuine, labelled churn outcome (the "Churn" column), which is exactly the target variable this project needs.
- It is one of the most widely used academic and industry benchmark datasets for churn prediction, meaning the results of this project can be compared against a large body of existing work.
- It contains a realistic mix of feature types — demographic (gender, SeniorCitizen), account information (tenure, Contract, PaymentMethod), and service usage (InternetService, StreamingTV, TechSupport) — which supports meaningful feature engineering.
- It is publicly and freely available, well documented, and does not require special authorization, which fits the scope of a college/internship project.
- Its size (~7,000 rows) is large enough to train and validate a machine learning model, yet small enough to process on a standard laptop without needing distributed computing.

### 1.3 Criteria Used to Judge Dataset Suitability

- Relevance: the data must directly relate to customer churn/retention in a service-based business.
- Target availability: a clear, labelled outcome column must exist (churned vs. retained).
- Volume: enough records to support train/validation/test splits without overfitting.
- Feature diversity: a mix of numerical and categorical fields to allow meaningful preprocessing and modelling.
- Data quality and documentation: column meanings should be documented, and the source should be reputable.
- Licensing/accessibility: the dataset must be free to use for academic purposes and legally shareable in the project's GitHub repository.

## 2. Data Extraction

### 2.1 How the Data Will Be Obtained and Loaded

- The CSV file has been downloaded from the source described in Section 1 and added to this project's GitHub repository at Week-2/data/WA_Fn-UseC_-Telco-Customer-Churn.csv, so the exact data used is version-controlled alongside this document.

- Python's pandas library will be used to load the file into a DataFrame using pd.read_csv().

```
import pandas as pd

df = pd.read_csv("Week-2/data/WA_Fn-UseC_-Telco-Customer-Churn.csv")
print(df.shape)
df.head()
```

### 2.2 Possible Problems During Acquisition

- Incorrect file path or file not found if the folder structure differs between environments.
- Encoding issues (e.g., non-UTF-8 characters) causing read errors.
- Columns being loaded with the wrong data type by default — for example, TotalCharges is stored as text because a few rows contain blank spaces instead of numbers.
- Extra whitespace or hidden characters in column names or values.
- Version mismatches if the dataset is updated on Kaggle after the project has already been downloaded, causing inconsistent results between team members.
- Network/download failures if the Kaggle API is used without proper authentication (kaggle.json credentials).

### 2.3 Confirmation on the Actual Dataset

The dataset has already been downloaded and loaded once to confirm it matches expectations before proceeding to full preprocessing:

- Shape: 7,043 rows × 21 columns — matches the expected size described in Section 1.
- Target distribution: Churn = "No" for 5,174 customers and Churn = "Yes" for 1,869 customers, confirming the target is present and usable, though moderately imbalanced (about 73% / 27%).
- Data type check: TotalCharges loads as an object (text) column rather than numeric, confirming the known issue described in Sections 3 and 4 — a small number of rows contain blank/whitespace values instead of numbers.
- Null check: df.isnull().sum() reports zero nulls, which confirms those problem values are hidden whitespace strings rather than true NaN entries — this is exactly why the validation step in Section 3 must check for blank/whitespace values specifically, not rely on isnull() alone.

## 3. Data Validation

Before any cleaning is performed, the raw dataset will be systematically checked to understand its structure and quality.

- **Rows and columns:** confirm the dataset shape using df.shape and df.info() to verify that all 7,043 rows and 21 columns have loaded correctly.
- **Data types:** review df.dtypes to identify columns that were loaded with an incorrect type (e.g., TotalCharges as object instead of float).
- **Missing values:** use df.isnull().sum() to count nulls per column, and additionally check for blank strings or whitespace-only entries that pandas would not flag as NaN.
- **Duplicate records:** use df.duplicated().sum() to identify fully duplicated rows, and check customerID for duplicate identifiers.
- **Invalid / inconsistent values:** use df[column].value_counts() on categorical columns to look for inconsistent labels (e.g., "No" vs. "No internet service" being treated as separate categories when they may need to be consolidated).
- **Outliers:** use df.describe() and boxplots on numerical columns (tenure, MonthlyCharges, TotalCharges) to detect unusually high or low values that may distort the model.

## 4. Data Cleaning

- **Missing values:** the blank entries in TotalCharges (associated with customers who have zero tenure) will be converted to NaN using pd.to_numeric(errors="coerce") and then either imputed with 0 (since tenure is 0) or with the column median, depending on what preserves the most information.

- **Duplicates:** any fully duplicated rows will be removed using df.drop_duplicates(), and customerID will be checked to ensure each customer appears only once.
- **Incorrect data:** columns will be corrected to their proper types (e.g., TotalCharges to float, SeniorCitizen to a categorical Yes/No if needed for consistency with other binary columns).
- **Categorical data preparation:** text values will be stripped of leading/trailing whitespace, standardized to consistent casing, and overlapping categories such as "No" and "No internet service" will be reviewed and, where appropriate, consolidated so the model does not treat them as unrelated categories.

## 5. Data Transformation

- **Encoding categorical variables:** binary categorical columns (e.g., Partner, Dependents, PhoneService, Churn) will be label-encoded (Yes/No → 1/0); multi-class categorical columns (e.g., InternetService, Contract, PaymentMethod) will be one-hot encoded using pd.get_dummies() or sklearn's OneHotEncoder.
- **Converting data types:** TotalCharges will be converted from object to float; SeniorCitizen (already 0/1) will be kept consistent with the other encoded binary fields.
- **Preparing features and target:** the DataFrame will be split into a feature matrix X (all columns except Churn and the non-predictive customerID) and a target vector y (the Churn column).
- **Scaling / normalization:** numerical features with different ranges — tenure, MonthlyCharges, and TotalCharges — will be scaled using sklearn's StandardScaler (or MinMaxScaler) so that no single feature dominates distance-based or gradient-based algorithms.

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X = df.drop(columns=["customerID", "Churn"])
y = df["Churn"]

num_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
scaler = StandardScaler()
X[num_cols] = scaler.fit_transform(X[num_cols])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

## 6. Python Tools and Justification

| Tool | Why It Is Used |
|---|---|
| Pandas | Primary tool for loading the CSV, inspecting structure (shape, dtypes, info), handling missing values, removing duplicates, filtering, and reshaping the data (get_dummies, groupby). Its DataFrame structure is the backbone of the entire preprocessing pipeline. |
| NumPy | Used underneath Pandas for efficient numerical operations, and directly for tasks such as identifying NaN patterns, computing statistics for outlier thresholds (e.g., IQR bounds), and working with arrays once data is converted for modelling. |
| Scikit-learn | Provides the preprocessing utilities needed to prepare the data for a model: LabelEncoder / OneHotEncoder for categorical encoding, StandardScaler for feature scaling, and train_test_split to create training and test sets in a reproducible way. |

# 7. Actual Customer Data Sample

The full dataset used for this project contains 7,043 customer records across 21 columns (see Section 1). The table below shows a real sample of 20 of those records — loaded directly from the CSV file with pandas — across 10 of the most relevant columns, to demonstrate that actual customer data (not placeholder/dummy data) is being used. The complete file, with all 21 columns and 7,043 rows, accompanies this submission in the project repository.

| customerID | gender | SeniorCitizen | Partner | tenure | InternetService | Contract | MonthlyCharges | TotalCharges | Churn |
|---|---|---|---|---|---|---|---|---|---|
| 7590-VHVEG | Female | 0 | Yes | 1 | DSL | Month-to-month | 29.85 | 29.85 | No |
| 5575-GNVDE | Male | 0 | No | 34 | DSL | One year | 56.95 | 1889.5 | No |
| 3668-QPYBK | Male | 0 | No | 2 | DSL | Month-to-month | 53.85 | 108.15 | Yes |
| 7795-CFOCW | Male | 0 | No | 45 | DSL | One year | 42.3 | 1840.75 | No |
| 9237-HQITU | Female | 0 | No | 2 | Fiber optic | Month-to-month | 70.7 | 151.65 | Yes |
| 9305-CDSKC | Female | 0 | No | 8 | Fiber optic | Month-to-month | 99.65 | 820.5 | Yes |
| 1452-KIOVK | Male | 0 | No | 22 | Fiber optic | Month-to-month | 89.1 | 1949.4 | No |
| 6713-OKOMC | Female | 0 | No | 10 | DSL | Month-to-month | 29.75 | 301.9 | No |
| 7892-POOKP | Female | 0 | Yes | 28 | Fiber optic | Month-to-month | 104.8 | 3046.05 | Yes |
| 6388-TABGU | Male | 0 | No | 62 | DSL | One year | 56.15 | 3487.95 | No |
| 9763-GRSKD | Male | 0 | Yes | 13 | DSL | Month-to-month | 49.95 | 587.45 | No |
| 7469-LKBCI | Male | 0 | No | 16 | No | Two year | 18.95 | 326.8 | No |
| 8091-TTVAX | Male | 0 | Yes | 58 | Fiber optic | One year | 100.35 | 5681.1 | No |
| 0280-XJGEX | Male | 0 | No | 49 | Fiber optic | Month-to-month | 103.7 | 5036.3 | Yes |
| 5129-JLPIS | Male | 0 | No | 25 | Fiber optic | Month-to-month | 105.5 | 2686.05 | No |
| 3655-SNQYZ | Female | 0 | Yes | 69 | Fiber optic | Two year | 113.25 | 7895.15 | No |
| 8191-XWSZG | Female | 0 | No | 52 | No | One year | 20.65 | 1022.95 | No |
| 9959-WOFKT | Male | 0 | No | 71 | Fiber optic | Two year | 106.7 | 7382.25 | No |
| 4190-MFLUW | Female | 0 | Yes | 10 | DSL | Month-to-month | 55.2 | 528.35 | Yes |
| 4183-MYFRB | Female | 0 | No | 21 | Fiber optic | Month-to-month | 90.05 | 1862.9 | No |

# 8. Preprocessing Workflow

## 8.1 Flow Diagram